Лекция 2 Контракты данных и предотвращение утечки

Курс: Управление процессами машинного обучения MLOps

Продолжительность: 2 академических часа

Связанная работа: ЛР2 «Контракт данных и предотвращение утечки»

Цель лекции

Показать, почему данные в ML-системе должны иметь явный договор между источником и потребителем, как определить момент доступности признака и как спроектировать хронологическое разбиение без утечки. После лекции студент должен уметь отличать data card от исполнимого контракта, сформулировать prediction timestamp, проверить point-in-time корректность и прочитать validation report как инженерное доказательство.

План и распределение времени

Время	Содержание	Действие преподавателя
0–10 мин	Связь с ЛР1 и постановка кейса	Восстановить путь от репозитория к данным и сформулировать прогноз рейса
10–25 мин	Данные как интерфейс	Разобрать владельца, производителя, потребителя и контракт
25–40 мин	Схема и семантика	Сравнить техническую и предметную корректность
40–55 мин	Момент прогноза и утечки	Построить временную шкалу признака и цели
55–70 мин	Хронологическое разбиение	Разобрать train, validation, test и преобразования
70–82 мин	Validation report и отказ	Показать положительный и отрицательный сценарии
82–90 мин	Инструктаж к ЛР2	Уточнить командные артефакты и индивидуальную защиту

1 Вход в тему

Начните с вопроса: «Если CSV успешно читается pandas, означает ли это, что модель можно безопасно обучать?» Ожидаемый ответ — нет. Файл может иметь правильные заголовки и типы, но содержать данные из будущего, иной смысл единиц, изменившееся кодирование категории или незаконно опубликованные сведения.

Свяжите тему с ЛР1. Репозиторий и воспроизводимая среда отвечают на вопрос «какой код и в каком окружении запущен». Контракт данных отвечает на вопросы «какие именно данные допустимы», «когда они были доступны» и «почему потребитель может им доверять». Без обеих частей воспроизводимость неполна.

Сформулируйте учебный кейс: за 60 минут до планового отправления система должна оценить, будет ли задержка не менее 15 минут. С этого момента любое поле нужно оценивать не по удобству для модели, а по доступности в конкретный момент времени.

2 Данные как инженерный интерфейс

Контракт данных — версионируемое соглашение между производителем и потребителем. Он описывает структуру, смысл, ограничения, временные свойства и реакцию системы на нарушение. Подчеркните, что контракт не гарантирует истинность каждой строки. Он делает ожидания явными и автоматически проверяемыми.

Разберите четыре стороны:

- источник формирует данные и сообщает об изменениях;
- владелец отвечает за разрешённое использование и качество;
- pipeline проверяет и преобразует данные;
- модель или сервис потребляет признаки в согласованном виде.

Полезный вопрос аудитории: «Кто должен исправить ошибку, если столбец исчез?» Ответ зависит от договора. Pipeline обязан быстро и понятно остановиться; исправление источника или адаптация потребителя — отдельное управляемое решение.

3 Data card и машинный контракт

Data card предназначена человеку. Она объясняет происхождение, назначение, лицензию, ограничения, чувствительные поля, известные смещения и порядок обновления. Машинный контракт предназначен автоматической проверке: обязательные столбцы, типы, nullability, диапазоны, категории, уникальность, формат времени и политика лишних полей.

Покажите, что документы дополняют друг друга. Правило `DepHour <= 23` исполнимо, но без описания единицы и часового пояса остаётся двусмысленным. Фраза «время отправления по местному времени аэропорта» понятна, но без проверки не остановит ошибочный pipeline.

Обсудите версионирование. Изменение описания — patch; совместимое необязательное поле — minor; изменение типа, имени или смысла обязательного поля — major. Это учебное соглашение, а не универсальный закон, но команда должна применять его последовательно.

4 Схема не равна семантике

Разделите проверки на уровни:

1. Структура: столбец присутствует, тип преобразуем, формат даты корректен.
2. Значения: диапазон, допустимая категория, пропуск, дубликат.
3. Взаимосвязи: фактическое время не раньше планового без объяснимой причины, аэропорты вылета и прилёта не совпадают для обычного рейса.
4. Распределения: доля пропусков или класс задержки резко изменились.
5. Временная корректность: значение было известно к prediction timestamp.

Сделайте акцент: первые два уровня часто дают жёсткий отказ, распределительные проверки могут начинаться с предупреждения. Команда должна объяснять выбранную реакцию, а не превращать любое отличие в падение системы.

5 Момент прогноза и временная линия

Нарисуйте на доске линию: планирование рейса → момент прогноза T−60 → отправление → прибытие → появление `ArrDelay`. Для каждого признака нужны два времени:

- event time — к какому событию относится значение;
- available time — когда значение фактически стало доступно системе.

Признак корректен, если available time не позже prediction timestamp. Например, плановый час отправления обычно известен заранее. Фактическое время отправления, итоговая задержка прибытия и статистика, рассчитанная с использованием будущих рейсов, недоступны.

Объясните разновидности утечки:

- target leakage — признак прямо или косвенно содержит ответ;
- look-ahead leakage — преобразование использует будущие наблюдения;
- split leakage — одна сущность или связанная запись попала в разные части;
- preprocessing leakage — нормализация или словарь категорий обучены на всех данных;

- operational leakage — offline-признак нельзя получить в реальном сервисе к требуемому времени.

Мини-задание: предложите поля `ScheduledDepTime`, `ActualDepTime`, `WeatherObservedAt`, `ArrivalDelay`, «средняя задержка аэропорта за сутки». Студенты должны задать уточняющие вопросы о времени доступности. Правильный ответ не всегда «можно» или «нельзя»; иногда нужно изменить окно агрегации.

6 Хронологическое разбиение

Для рейсов случайное перемешивание создаёт оптимистичную оценку, потому что будущее смешивается с прошлым. Используется последовательность train → validation → test. Train обучает параметры модели и преобразований. Validation нужен для выбора модели, признаков и порога. Test имитирует окончательную проверку на будущем периоде и не должен многократно влиять на решения.

На доске зафиксируйте инвариант:

```
max(train_time) < min(validation_time) <= max(validation_time) < min(test_time)
```

Границы задаются календарными датами и числами строк. Если распределение классов неудобно, запрещено решать проблему переносом отдельных будущих строк в train. Нужно выбрать более длинное окно, изменить постановку или явно описать ограничение.

Все обучаемые преобразования fit только на train. На validation и test применяется transform. Это относится к заполнению пропусков, нормализации, отбору признаков, target encoding и формированию словарей.

7 Validation report как доказательство

Отчёт должен позволять другому человеку понять, что именно проверено. Минимальные поля: версия контракта, логический идентификатор набора, путь или URI, SHA-256, число строк и столбцов, границы периодов, результат каждого правила и общий статус.

Покажите два запуска. Первый проходит и завершается кодом 0. Во втором дата имеет неверный формат или строки переставлены; отчёт называет правило, а процесс возвращает ненулевой код. Отдельно проговорите: перехват исключения с последующим `exit 0` делает CI недостоверным.

SHA-256 связывает отчёт с байтами файла, но не доказывает качество, лицензию или отсутствие смещения. Это идентификатор, а не сертификат доверия.

8 Командная работа и индивидуальная ответственность

В команде 2–5 человек можно распределить роли data steward, contract engineer, quality engineer, leakage reviewer и documentation reviewer. Роль не является доказательством. Нужны собственный commit или review, воспроизводимый запуск и объяснение решения.

На защите каждый студент запускает валидатор, показывает SHA-256 и границы периодов, создаёт одно контролируемое нарушение и объясняет доступность выбранного признака за 60 минут до отправления. Одинаковый текст evidence card у нескольких участников не принимается.

9 Вопросы для проверки понимания

1. Почему корректный тип столбца не доказывает корректный смысл?
2. Чем data card отличается от `data_contract.json`?
3. Почему `ArrDelay` допустим для построения цели, но запрещён как признак?
4. Чем event time отличается от available time?
5. Почему нельзя fit нормализатор на объединённых train и test?
6. Что доказывает SHA-256 и чего он не доказывает?
7. Когда изменение контракта требует major-версии?
8. Почему test-период нельзя использовать для регулярного выбора гиперпараметров?

10 Завершение лекции

Верните аудиторию к исходному вопросу. Файл становится пригодным для ML-системы не тогда, когда он просто читается, а когда его происхождение, назначение, схема, временная доступность и правила отказа зафиксированы и воспроизводимо проверяются. ЛР2 должна создать этот барьер качества; ЛР3 затем свяжет проверенный набор с Git-состоянием через DVC.

Чек-лист преподавателя

- На временной линии явно отмечены prediction timestamp и появление цели.
- Разобраны event time и available time.
- Показано хронологическое train/validation/test без перемешивания.
- Продемонстрированы успешный и отказной validation report.
- Уточнено, что основной набор BTS не публикуется без проверки условий использования.
- Озвучены общий командный результат и индивидуальные доказательства.